

職務経歴書

2026年6月24日現在

氏名: 近藤智広(コンドウ トモヒロ)

■職務要約

約10年のWebアプリケーション開発経験を持ち、Ruby on Railsを中心としたバックエンド開発が専門領域です。物件・商品・会員データの管理、一覧・検索API、MySQLチューニング、RSpecによるテスト、AWS上での運用まで一貫して担当してきました。2週間スプリントを前提としたアジャイル開発の中で、API設計、DB設計、アーキテクチャ改善、性能チューニング、コードレビューを行い、生成AI（OpenAI API）をプロダクトに組み込む機能開発にも携わっています。

TypeScript / React / Vue.js、Laravel、Node.js、Elasticsearch / Algolia、OpenAPI、BigQueryは補助スキルとして案件に応じて活用。SaaS型の業務管理・分析・EC支援など、顧客が日常業務で使うBtoBプロダクトの開発経験が多く、不動産・住宅領域のようにデータ量と検索性能が重要なドメインでも、ユーザー課題と運用負荷を踏まえたバックエンド改善を重視してきました。数名規模のチームでのレビュー補助や実装リード的役割はある一方、大規模チームのマネジメント専任経験は限定的です。

■キャリア成長の概要

2015年～2017年は実装・テスト・不具合修正を中心に基礎を習得。2018年～2020年はAPI実装、MySQL改善、テスト設計を担当し、レビュー補助を任されることが増えました。2020年～2021年はAWS移行案件でサブリーダー補佐（リリース確認、タスク取りまとめ）に加わりました。2021年～2023年はEC・受注管理SaaSで、一覧・検索・夜間バッチなどデータ負荷の高いバックエンド領域の設計・性能改善を担当。2023年～2024年は刷新案件でバックエンド主担当となり、OpenAPI整備、Redisキャッシュ、RSpec / PHPUnitを含むテスト整備を推進。2024年～2025年は分析SaaSと自社アプリで、検索性能改善、生成AI機能、モバイル向けAPI連携を担当。2021年頃から並行してRuby on Railsを用いたAPI開発・バッチ処理にも携わり、2026年以降はSaaS刷新案件でAPI設計・レビュー・CI/CD整備を中心に実装リード的役割を担っています。

■学歴・キャリア開始補足

2015年10月～2016年3月は在学中の長期インターンとして開発補助・テスト補助に従事。2016年3月に大学を卒業後、2016年4月より同社へ正社員として入社し、Webアプリケーション開発を本格的に担当しました。職務経歴上の実務開始は長期インターン参加時点から記載していますが、正社員としての就業開始は2016年4月です。

■職務経歴

2026年1月～現在 / 株式会社オヤテックジャパン（OyaTech Japan, Inc.）（契約／フルリモート）

Webシステム・SaaSの開発受託企業にて、契約エンジニア（実装リード的役割）として従事。5名規模のマーケティングオートメーションSaaS刷新案件で、Ruby on Rails / Laravel APIの設計・実装・レビュー、MySQL改善、AWS運用、GitHub ActionsによるCI/CD整備を担当。2週間単位のイテレーションで優先度を整理し、海外エンジニアを含むチームでのPRレビューと実装方針共有、クライアント向け技術進捗の報告補助を行う。

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【マーケティングオートメーションSaaS刷新・機能強化】（2026/01～現在） クライアント向けマーケティングオートメーションSaaSの刷新を担当。既存コードベースの現状分析から開始し、担当範囲のアーキテクチャ改善と機能の設計・実装を行う。 【業務内容】 - 既存コードベースの現状分析と改善項目の分類・優先順位付け - Ruby on Rails / Laravel APIのリファクタリング、新機能APIの設計・実装、OpenAPIドキュメント整備 - MySQLのスキーマ見直し、遅延クエリ調査、インデックス最適化 - Next.js / TypeScript による管理画面改修（必要範囲） - Terraformを用いたAWSインフラ（ECS, RDS, S3, CloudFront）の既存構成改修とIaC化 - GitHub ActionsによるCI/CDパイプラインの構築・改善 - 開発チーム（海外エンジニア含む）へのPRレビュー、実装方針の共有 - 実装リードとして、担当領域のスケジュール調整、進捗報告、クライアントとの技術面の調整補助 【課題・工夫】 ドキュメント整備が追いついていない領域があったため、既存コードの動作検証から着手。海外エンジニアとの非同期コミュニケーションを前提に、PRテンプレートやIssueテンプレートを整備し、認識齟齬を減らす工夫を実施。インフラ構成の見直しに伴い、TerraformでIaC化し、再現性と運用負荷を改善。 【工程別の担当】 現状調査：既存コードベースの現状分析、改善項目の分類、優先順位付けを実施。 設計：フロントエンドのコンポーネント構成、API設計、インフラ構成を要件に沿って再設計。 実装：Next.jsフロントエンド、Laravel API、Terraformインフラ改修を中心に担当。 テスト：単体テスト・結合テストの整備、CIパイプラインへの自動テスト組み込みを実施。 リリース・運用：段階的なリリース計画の策定と実行、運用ドキュメントの整備。 【実績】	【主軸】 Ruby on Rails, Laravel, MySQL, AWS 【補助】 TypeScript, Next.js, GitHub Actions, RSpec, Terraform	【規模】 5名（海外3名含む） 【役割】 契約エンジニア（実装リード） 【従事割合】 約80%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
・約3ヶ月で主要機能の刷新と本番運用体制を確立 ・インフラ構築時間を手動の数日からTerraform実行の約30分に短縮 ・CI/CD整備によりデプロイ頻度を週1回から週3〜4回に向上 ・海外メンバー向けにPRテンプレートとレビュー観点を共有し、認識齟齬の削減に貢献		

【今後の志向】 不動産・空間データのように、コンテンツと検索性能が事業価値に直結するSaaSで、Railsバックエンドの設計・運用改善に継続して携わりたいと考えています。生成AI機能や検索基盤の強化にも関心があり、ユーザーが日常業務で使うプロダクトの品質向上に貢献したいです。

2024年4月～2025年12月 / 株式会社Gonmura

お笑い・エンタメ領域の自社サービス開発に従事。フルスタックエンジニアとして実装・保守を担当し、担当期間では生成AI機能のプロダクト組み込み、一覧・検索APIの性能改善、モバイルアプリ向けAPI連携、2週間スプリントでの機能リリースを経験。設計整理、PRレビュー、週次進捗の共有補助を任される場面がありました。

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【Fanfun運用基盤・生成AIサポート機能改善】(2025/07～2025/12) 自社アプリ「Fanfun」の運用管理画面と、サポート向け生成AI検索機能の改善を担当。配信コンテンツ公開状態、会員プラン、イベント応募状況など、問い合わせが多い領域を整理し、運用担当者が調査しやすい管理導線とAI支援機能を開発。 【業務内容】 ・Ruby on Rails / Laravel APIでの会員状態、通知設定、イベント応募状況の取得・整形処理実装 ・OpenAI APIを用いたFAQ検索、回答候補提示、根拠リンク表示機能の設計・実装 ・Next.js / TypeScript による設定画面、状態表示、ヘルプ導線のUI改善 ・TerraformによるCloudFront / WAF設定変更、PRレビュー、ステージング検証 ・サポート担当者へのヒアリング、問い合わせ分類、改善対象画面の優先順位付け ・AI出力の確認観点、Issueテンプレート、運用手順書の整備 ・同僚エンジニアのPRレビュー、実装方針相談、問い合わせ調査手順の共有 ・Fanfun領域の週次進捗共有、関係者間の調整補助 【課題・工夫】 配信公開や通知設定、イベント応募状況はファンから見えにくく、問い合わせ時もサポート担当者が複数画面を確認する必要がありました。そのため、ファン向け画面では「現在の状態」「次に必要な操作」「反映に時間がかかる理由」を短い文言で表示し、サポート向けには会員状態・設定値・ログ確認リンクをまとめて確認できる設計にしました。AI回答候補については、誤回答を避けるため、回答文だけでなく参照元FAQ、確認すべき管理画面、ユーザーへ案内する前のチェック項目を併記する方針としました。 【工程別の担当】 要件整理：サポート担当者へのヒアリングを行い、問い合わせ件数が多い通知未達、イベント応募、会員プラン、コンテンツ公開状態を優先対象として整理。 設計：ファン向け画面とサポート向け画面で必要な情報粒度を分け、APIレスポンス、FAQ検索結果、根拠リンク、確認観点的表示方針を設計。 実装：Next.js画面、Laravel API、OpenAI API連携、Terraform変更を担当し、ステージング環境で設定状態ごとの表示を確認。 テスト：通知未達、イベント応募期限切れ、会員停止、コンテンツ非公開などの異常系を洗い出し、AI回答候補が過剰に断定しないことを確認。 リリース・運用：サポート担当者向けに確認手順を共有し、問い合わせ対応時のフィードバックを次の改善Issueへ反映。 【実績】 設定手順で離脱・問い合わせが多かった箇所を画面内で説明できるよう改善。通知・イベント応募関連の月次問い合わせはリリース前後の比較で約18%減少し、サポート担当者の初動調査時間も1件あたり平均15分程度短縮。AI回答は人が確認する前提で根拠リンクと確認観点を併記し、誤回答を避けながらサポート初動調査の効率化に貢献。	【主軸】 TypeScript, Next.js, Laravel, AWS 【補助】 OpenAI API, Terraform	【規模】 10名 【役割】 主担当エンジニア（Fanfun領域） 【従事割合】 約65%
【ライブ公演・チケット管理プラットフォーム改善】(2025/04～2025/12) お笑いライブ公演のチケット販売、出演者調整、会場・配信枠管理に関わる管理機能と通知機能の改善を担当。運営メンバーが公演案件の状態を素早く判断できるよう、出演確認・販売開始・入金・配信準備状況を一面で追跡できる管理導線を整備。 【業務内容】 ・Laravel / TypeScript による公演案件管理、出演者確認、審査画面の改修 ・Node.js Webhook連携による決済・販売ステータス更新処理の実装 ・通知テンプレート、送信履歴、再送処理の管理機能追加 ・Sentry / CloudWatchを用いた決済・通知関連エラーの原因調査と再発防止 ・サービス運営メンバーからの改善要望整理、仕様化、受入確認支援 ・権限別の表示制御、監査ログ、操作履歴の確認観点整理 ・チケット販売・配信準備まわりの例外ケースを洗い出し、テストケースへ反映 ・機能単位の実装方針整理、PRレビュー、リリース前確認観点的作成 【課題・工夫】 公演状態が「調整中」「出演確認中」「販売準備中」「販売中」「配信準備中」など複数に分かれ、画面ごとに表示が分散していたため、運営メンバーが次アクションを判断しづらい状態でした。状態遷移を一	【主軸】 TypeScript, Laravel, Node.js, MySQL 【補助】 Vue.js, Sentry	【規模】 7名 【役割】 主担当エンジニア 【従事割合】 約60%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
覧化し、ステータス、担当者、最終更新日時、次に必要な確認項目を同一画面で確認できるよう設計しました。また、チケット販売・配信領域では例外ケースが多いため、Webhook再送、通知失敗、手動修正時の監査ログを重視しました。 【工程別の担当】 要件整理：運営メンバーの公演確認手順を観察し、出演確認、販売開始、入金確認、配信準備で重複している確認作業を抽出。 設計：公演ステータスの状態遷移、権限別の操作範囲、通知タイミング、操作履歴の保持方針を整理。 実装：Laravel管理画面、Vue.jsコンポーネント、Node.js Webhook処理、通知テンプレート管理機能を実装。 テスト：決済成功・失敗、Webhook遅延、通知再送、手動ステータス変更、権限不足時の表示など例外系を重点確認。 リリース・運用：既存公演への影響を避けるため段階的に機能を有効化し、運営メンバーの受入確認後に通知文言を調整。 【実績】 ステータス表示、検索条件、通知文言を整理し、運営メンバーの確認作業を削減。公演確認にかかる時間は1件あたり平均8～10分から5分前後に短縮。販売・配信準備の状態遷移を一覧で追えるようにしたことで、案件ごとの次アクションを判断しやすくした。ファンからの「次に何をすべきか分からない」という問い合わせも対象カテゴリで約20%減少。		
【公演集客・データ分析SaaS機能改善】（2024/10～2025/06） 告知効果の可視化、保存リスト、検索・CSV出力機能を持つ分析SaaSにて、一覧・検索APIの性能改善とデータ活用導線の整理を担当。物件・商品マスタに近い「分析対象データの管理・検索」領域で、画面・API・DBを横断して対応。 【業務内容】 ・Node.js / Ruby on Rails APIのページング、フィルタ、レスポンス形式の見直し ・Aurora MySQLのSQL改善、複合インデックス追加、Redisキャッシュ設計見直し ・検索レスポンス改善のため、Elasticsearch導入検討と既存SQLチューニングの比較評価 ・GitHub Actions / AWS CodePipelineによるテスト・デプロイ手順の改善 ・PdM、デザイナー、CS担当と問い合わせ内容を分析し、改善優先度を整理 ・公演別の分析条件保存、再検索、CSV出力時の非同期処理・進捗表示を改善 ・API利用者向けのレスポンス仕様、エラーコード、レート制限説明を整理 ・検索・CSV出力領域のレビュー観点整理、PRテンプレートへの性能確認項目追加 【課題・工夫】 公演告知チャンネルやSNS投稿の分析条件が増えるにつれて、画面操作、APIレスポンス、DB負荷のいずれにも影響が出る状態でした。単純に画面だけを修正するのではなく、運営担当者がどの条件を組み合わせるのか、CSV出力をどのタイミングで行うのか、API利用者がどの項目を必要としているかを確認し、画面・API・DBの役割を整理しました。大量データ取得では同期処理に寄せすぎないように、非同期出力、進捗表示、キャッシュ利用を組み合わせました。 【工程別の担当】 要件整理：CSへの問い合わせ、利用頻度の高い分析条件、CSV出力の利用シーンを確認し、改善対象を分析体験・保存リスト・API連携に分解。 基本設計：画面状態、分析条件保存、APIレスポンス、ページング、非同期CSV出力、キャッシュの責務を整理。 詳細設計：SQL実行計画、複合インデックス、Redisキー設計、エラーコード、レート制限、API利用者向け説明項目を定義。 実装：Next.js画面、Node.js API、CSV出力ジョブ、DBクエリ改善、GitHub Actionsの性能確認項目を担当。 テスト：大量データ、条件組み合わせ、CSV出力中断、APIエラー、権限不足、キャッシュ不整合を確認。 リリース後：問い合わせ内容を確認し、空状態・エラー表示・ヘルプ文言を継続的に調整。 【実績】 保存リストとCSV出力の導線を整理し、定型的な集客分析作業の操作数を平均6クリックから3～4クリックへ削減。大量データ検索時の主要API応答時間は平均1.8秒前後から1.1秒前後へ改善し、CSV出力は非同期化によりタイムアウト発生を月数件からほぼ0件に抑制。問い合わせ内容をもとにUI文言、空状態、エラー表示を見直し、初回利用者にも意図が伝わる画面に調整。	【主軸】 TypeScript, Next.js, Node.js, Aurora MySQL 【補助】 Redis, GitHub Actions	【規模】 13名 【役割】 主担当エンジニア 【従事割合】 約60%
【AI活用ワークフロー整備】（2024/04～2024/09） 開発支援AIを活用したIssue整理、実装補助、レビュー観点作成のワークフローを整備。Fanfunや公演管理アプリなどエンタメ系プロダクト開発のGitHub運用にAI活用を組み込み、機密情報の扱い、出力確認、レビュー責任を明確にした上で、チームで再利用できるテンプレートを作成。プロダクト向けAI API連携は、後続のFanfun運用基盤・AIサポート機能改善で担当。 【業務内容】 ・GitHub Copilot / Claude / Clineを用いた実装補助、差分説明、レビュー補助の検証 ・Issue分解、PR説明、テスト観点作成、リファクタリング方針比較のテンプレート化 ・AI利用時の機密情報、著作権、セキュリティ観点のチェックリスト作成 ・チーム向け共有会の実施、共有資料の整備 【課題・工夫】 AI活用は生産性向上につながる一方で、出力の正確性や機密情報の扱いに注意が必要でした。そこで、利用してよい情報・避けるべき情報、生成結果を採用する前の確認観点、レビュー時に人が見るべきポイントを明文化しました。短期間の取り組みだったため、まずはIssue分解、PR説明、テスト観点作成	【主軸】 TypeScript, GitHub 【補助】 Claude, Copilot	【規模】 3名 【役割】 開発フロー改善の試行担当 【従事割合】 約30%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
など効果が見えやすい用途に絞って導入しました。 【工程別の担当】 調査：既存開発フローの中でAI活用しやすい作業とリスクが高い作業を分類。 設計：Issue、PR、レビュー、テスト観点作成のテンプレートを作成。 展開：小規模チームで試行し、利用時の注意点と改善点を共有資料へ反映。 運用：AI出力を人が確認する前提で、レビュー観点と禁止事項を継続更新。 【実績】 定型的な実装準備、PR説明、レビュー観点作成の品質を標準化。試行対象の小規模Issueでは、PR説明・テスト観点作成にかかる時間を平均30～40%短縮。AI出力の採用判断を人が行う前提を徹底し、レビュー差し戻しも導入前と比較して約15%減少した。		

【退職理由】 自社サービス開発の経験を活かし、担当範囲を明確にした契約ベースの開発案件へ移行しました。
--

2021年4月～2024年3月 / 株式会社Proteinum

ECコンサルティング・運営代行、EC支援SaaS、D2C（サプリメント等）事業に関わるWebサービス開発に従事。入社後1年程度は保守運用・小規模改修が中心で、その後フルスタック実装の比重が増えました。2023年頃からは刷新案件でバックエンド主担当となり、担当機能の設計整理やレビュー、顧客向け進捗共有の補助を行いました。

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【D2Cサプリメントアプリ向けAPI・CI/CD整備】（2024/02～2024/03） 自社D2Cサプリメントアプリ向けAPIとCI/CD整備に参画。短期間でAPI仕様、モック、認証、通知、デプロイ手順を整備する必要があり、モバイルチームと並行開発できるようAPI契約と検証環境を優先して構築。 【業務内容】 ・ Node.js を用いたAPIゲートウェイ、認証、エラーレスポンス設計 ・ モバイルチームと協力したOpenAPI仕様策定、モック駆動開発の導入 ・ AWS ECS / ALBを用いた検証環境構築、ヘルスチェック、オートスケーリング設定 ・ GitHub ActionsによるLint、テスト、Docker Build、デプロイの自動化 ・ 短期案件の実装タスク分割、APIレビュー、モバイル側との仕様調整窓口 【課題・工夫】 期間が短く、モバイル側の実装とAPI側の実装を並行して進める必要があったため、詳細実装より先にOpenAPI仕様とモックを整備しました。認証エラー、入力不備、メンテナンス中などのレスポンス形式を先に固定し、モバイル側が画面実装を止めずに進められるようにしました。 【実績】 API仕様の共有とモック駆動開発により、モバイル側との認識差分を実装初期に検出できる体制を構築。API仕様起因の手戻りは結合テスト期間中に約30%減少し、検証環境への反映作業も手動30分程度からGitHub Actions経由で10分以内に短縮。	【主軸】 TypeScript, Node.js, AWS 【補助】 Flutter連携, OpenAPI	【規模】 8名 【役割】 API設計・CI/CD担当 【従事割合】 約35%
【EC支援SaaS業務管理刷新】（2023/02～2024/01） ECモール運用・広告分析を支援するSaaS型業務管理システムのリニューアルPJにてバックエンド設計・実装を担当。既存機能の仕様がドキュメント化されていない箇所が多かったため、現行画面・DB・運用手順を確認しながらLaravel APIとReact画面へ段階的に移行。 【業務内容】 ・ PHP / Laravel を用いた新API設計、認証・権限、バリデーション実装 ・ OpenAPIドキュメント整備、API仕様レビュー、フロントエンド連携確認 ・ React SPAのコンポーネント分割、フォーム状態管理、エラー表示方針整理 ・ Redisキャッシュ戦略を構築し、一覧・検索系APIのDB負荷を軽減 ・ 移行対象機能の依存関係、権限、バリデーション差分を一覧化 ・ 旧画面と新画面の並行稼働期間を考慮したリリース単位の分割 ・ コードレビュー、週次共有会、移行差分リスト作成による実装方針の統一 ・ バックエンド領域のタスク分割、レビュー、メンバーへの実装方針説明 ・ 顧客との進捗共有、開発スケジュール調整、QA日程調整の補助 【課題・工夫】 既存システムは長期間運用されており、画面上の挙動とDB上の状態が暗黙的に結びついている箇所が多ありました。まず現行画面の操作、DB更新、運用担当者の手順を確認し、機能ごとに「そのまま移行する仕様」「改善する仕様」「廃止候補」を分類しました。移行期間中は旧画面と新画面が並行稼働するため、データ整合性、権限、通知タイミングが変わらないように差分表で管理しました。 【工程別の担当】 要件整理：既存画面、DB更新内容、運用担当者の手作業を確認し、移行対象機能と廃止候補機能を分類。 基本設計：Laravel API、React SPA、認証・権限、キャッシュ、OpenAPI管理方針を整理。 詳細設計：一覧・検索・更新系APIのレスポンス形式、フォーム入力、バリデーション、エラー表示を定義。 実装：API、Reactコンポーネント、Redisキャッシュ、OpenAPIドキュメント、移行差分リストを担当。 テスト：旧画面との出力差分、権限別表示、検索条件、更新時の整合性、キャッシュ更新タイミングを確認。 リリース：機能単位で段階移行し、旧画面との並行稼働期間中に問い合わせ内容と不具合を整理。	【主軸】 PHP, Laravel, React, MySQL, Redis 【補助】 GitHub Actions, OpenAPI	【規模】 9名 【役割】 バックエンド主担当 【従事割合】 約70%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【実績】 既存仕様差分を約80項目一覧化し、移行時の認識違いによる手戻りを削減。API、画面、DB、運用手順の差分を追跡できるようにしたことで、レビュー指摘の再発件数を約25%削減。検索・一覧系APIの平均レスポンスは約1.5秒から0.9秒前後へ改善し、OpenAPI整備によりフロントエンドとの仕様確認時間も短縮。		
【ECモール受注・会員管理システム機能拡張】（2021/11～2023/01） ECコンサル案件および自社D2C事業向けの受注・会員管理システムの機能拡張と性能改善を担当。受注枠、会員ステータス、通知履歴、夜間集計バッチの処理が複雑化していたため、運用上の確認手順とシステム処理の両面から改善を実施。 【業務内容】 ・Node.js / TypeScript を用いた受注API、通知API、バッチ処理の改修 ・夜間バッチ処理の並列化、リトライ制御、エラーログ出力の改善 ・MySQLの遅延クエリ調査、インデックス最適化、集計SQLの見直し ・Vue.js管理画面の検索条件、ステータス表示、リアルタイム更新機能追加 ・受注枠の重複、キャンセル、再通知など例外パターンの整理 ・JMeterによる負荷テストを実施し、改善候補と運用上の注意点を整理 ・運用担当者向けにバッチ失敗時の確認手順と復旧手順を作成 ・受注・会員管理領域のレビュー担当として、SQL・バッチ・管理画面改修の確認を実施 【課題・工夫】 受注・会員管理は日中の画面操作と夜間バッチの両方で同じデータを扱うため、処理タイミングのずれによる不整合が課題でした。受注枠、キャンセル、会員ステータス、通知履歴の関係を整理し、バッチ処理のリトライ条件とエラー時の確認手順を明確化しました。管理画面では、運用担当者が問い合わせを受けた際に必要な情報へすぐ到達できるよう検索条件と表示順を見直しました。 【工程別の担当】 要件整理：受注枠、会員ステータス、通知履歴、キャンセル処理の運用フローを確認し、障害時に困る確認作業を抽出。 設計：API、バッチ、管理画面、通知処理の責務を分け、夜間バッチ失敗時の再実行・確認手順を定義。 実装：Node.js API、TypeScriptバッチ、Vue.js管理画面、通知履歴表示、検索条件追加を担当。 性能改善：遅延クエリを調査し、インデックス追加、集計SQL見直し、バッチ並列化を実施。 テスト：JMeterでピーク時の受注操作を想定した負荷確認を行い、処理時間とエラー発生箇所を確認。 運用改善：バッチ失敗時の確認手順、復旧手順、問い合わせ時の確認画面を整理。 【実績】 夜間バッチの処理時間を約95分から約55分へ短縮し、失敗時の一次調査手順も整備。受注枠・会員状態・通知履歴を同一画面で確認できるようにしたことで、運用担当者の問い合わせ確認時間を1件あたり約10分削減。負荷テストではピーク想定アクセス時の主要APIエラー率を1%未満に抑え、次期改善候補も整理。	【主軸】 TypeScript, Node.js, MySQL 【補助】 Vue.js	【規模】 7名 【役割】 主担当エンジニア（後期はレビュー補助） 【従事割合】 約55%
【EC支援SaaS保守運用・改善】（2021/04～2021/10） 既存EC支援SaaSの保守運用と機能改善を担当。障害調査、問い合わせ対応、管理画面改修、クライアント案件データ修正依頼への対応を通じて、既存サービスの構造と運用課題を把握。 【業務内容】 ・障害対応、問い合わせ対応、調査用SQL作成を通じたシステム構造の把握 ・Laravel管理画面の検索条件、一覧表示、CSV出力機能の改修 ・SQLチューニング、遅延クエリ特定、不要なN+1取得の改善 ・Sentry / CloudWatchの通知整理、エラー分類、一次調査手順の作成 【課題・工夫】 保守運用フェーズでは、機能開発よりも「早く正確に状況を把握すること」が重要でした。問い合わせ内容を分類し、どの画面・ログ・SQLを確認すべきかを手順化することで、属人的な調査を減らしました。また、通知が多すぎて重要なエラーを見落とす状態を避けるため、Sentryの通知条件とCloudWatchログの確認観点を整理しました。 【工程別の担当】 調査：問い合わせ内容、障害発生画面、ログ、DB状態を確認し、一次切り分けを実施。 改修：Laravel管理画面、CSV出力、検索条件、SQL改善など小中規模の改修を担当。 検証：本番データに近い条件で検索結果、権限、出力件数、エラー再現性を確認。 運用：Sentry通知、CloudWatchログ、調査SQL、手順書を整備し、同種問い合わせに再利用できる形に整理。 【実績】 障害調査時に確認すべきログ、画面、SQLを整理し、一次調査の属人化を軽減。定型問い合わせの一次調査時間は平均20分前後から10～12分程度へ短縮。管理画面の検索性改善により、運用担当者の確認作業も月あたり約8時間削減した。	【主軸】 PHP, Laravel, MySQL 【補助】 Sentry, CloudWatch	【規模】 5名 【役割】 メンバー 【従事割合】 約50%
【退職理由】 ECコンサル・運営代行から、よりユーザーの反応を見ながら継続改善できる自社D2C・自社サービス開発へ軸足を移したいと考え退職しました。		

2015年10月～2021年3月 / ヴィンテージIT株式会社

2015年10月～2016年3月は在学中の長期インターンとして参画し、2016年4月より正社員として入社。医療・福祉領域に特化したソフトウェアメーカーにて、初期は先輩エンジニアの指示のもと、テスト、軽微な改修、ドキュメント整備、障害票調査を担当。その後、医療機関向け施設基準管理、社会福祉協議会向け相談・自立支援業務管理、利用者向けモバイルアプリ、クラウド基盤AWS移行などを経験し、Webアプリケーション開発の基礎を段階的に習得。後半は小規模タスクの取りまとめ、レビュー補助、リリース確認も担当。

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【医療・福祉クラウド基盤AWS移行】（2019/11～2021/03） 医療機関向け施設基準管理システムと社会福祉協議会向け業務管理クラウドのAWS移行PJに参加。オンプレミス環境で稼働していたLaravelアプリケーションをAWS ECS / RDS / S3 / CloudFront構成へ移行するため、アプリ改修、データ移行、検証、本番切替支援を担当。 【業務内容】 ・ PHP / Laravel アプリケーションの環境依存処理、帳票・添付ファイル保存処理の改修 ・ AWS ECS / RDS / S3 / CloudFront 構成での動作検証、設定確認 ・ GitHub ActionsによるCI/CDパイプライン整備、環境変数管理、リリース手順作成 ・ 相談記録・施設基準データの移行バッチ開発、移行前後の件数・整合性チェック実装 ・ 帳票・添付ファイルのS3移行、URL変換、配信確認 ・ 小規模タスクの進捗確認、レビュー補助、切替当日の確認項目整理 ・ ジュニアメンバーの作業分担確認、移行チェックリストのレビュー補助（サブリーダー補佐として） 【課題・工夫】 移行対象には相談記録、施設基準関連帳票、添付ファイル、利用者情報などが含まれ、単純な環境移設だけではなく、ファイル参照先やバッチ処理の見直しが必要でした。移行前後でデータ件数、帳票表示、通知メール、管理画面の検索結果が一致するかを確認するため、チェックリストと検証SQLを作成しました。切替当日は担当ごとの確認範囲を分け、問題発生時に戻せるようロールバック手順も事前に整理しました。 【工程別の担当】 現状調査：オンプレミス環境のディレクトリ構成、バッチ実行タイミング、ファイル保存先、DB接続設定を確認。 移行設計：ECS / RDS / S3 / CloudFront構成でのアプリ変更点、環境変数、ログ出力、ファイル配信 방식을整理。 実装：Laravelアプリのファイル保存処理、S3移行バッチ、URL変換処理、GitHub Actionsのデプロイ処理を担当。 検証：移行前後の件数、帳票表示、通知メール、相談記録、管理画面検索、バッチ結果をチェックリスト化して確認。 リリース：本番切替手順、担当者別確認項目、ロールバック条件を整理し、切替当日の確認を支援。 運用移行：移行後のログ確認、バックアップ確認、障害時の一次確認手順をドキュメント化。 【実績】 移行手順書、ロールバック手順、切替時の確認項目を整備し、約12万件のデータ移行と帳票・添付ファイル移行を大きな障害なく完了。切替当日の確認項目を約60項目に整理し、移行後のログ確認、帳票配信、通知メールなど運用観点の確認も担当。デプロイ手順整備により、リリース作業時間は約40分短縮。	【主軸】 PHP, Laravel, AWS 【補助】 Docker, GitHub Actions	【規模】 9名 【役割】 サブリーダー補佐（移行案件内） 【従事割合】 約70%
【利用者向けヘルスケアアプリ新規開発】（2018/10～2019/01） 利用者・家族向けヘルスケアアプリ（iOS/Android）の新規開発PJに参画。Node.js / Express によるBFF実装を主軸とし、モバイル画面はReact Nativeで実装。認証、お知らせ通知、服薬・予約リマインド、アプリ審査対応まで経験。 【業務内容】 ・ React Native を用いたログイン、お知らせ一覧、相談予約、設定画面の実装 ・ Node.js / Express によるBFF設計、API集約、レスポンス整形処理の実装 ・ Firebase Authentication / Cloud Messagingを用いた認証・プッシュ通知基盤構築 ・ クラッシュログ確認、端末差分調査、ストア申請用スクリーンショット作成 ・ オフライン時の表示、通信エラー、再ログイン時の状態復元処理の改善 ・ App Store / Google Play リリース対応、審査指摘への修正対応 【課題・工夫】 医療・福祉系アプリでは端末・OS・通信状態による差分が大きく、利用者が高齢の場合も多いため、状態表示の分かりやすさが重要でした。クラッシュログ、端末情報、操作手順をセットで残す運用にし、認証切れや通信エラー時でも利用者が状態を理解できるようメッセージと再試行導線を調整しました。 【工程別の担当】 要件整理：ログイン、お知らせ、相談予約、通知、設定など初回リリースに必要な画面を整理。 設計：React Native画面構成、BFFのAPI集約方針、Firebase認証・通知の利用方針を確認。 実装：主要画面、BFF、プッシュ通知、通信エラー時の表示、再ログイン時の状態復元処理を担当。 テスト：iOS / Androidの端末差分、通信不安定時、プッシュ通知、認証切れ、ストア審査観点を確認。 リリース：ストア申請、審査指摘対応、リリース後のクラッシュログ確認と軽微な改善を実施。 【実績】 初回リリース後のクラッシュログと問い合わせ内容をもとに改善を継続。主要クラッシュはリリース後1ヶ月で6件から1件まで減少し、通知導線と初回設定画面の改善により初回設定完了率は約12ポイント向上。アプリ審査指摘にも2営業日以内で対応した。	【主軸】 TypeScript, Node.js 【補助】 React Native（当時）	【規模】 6名 【役割】 メンバー / API担当 【従事割合】 約50%
【社会福祉協議会向け業務管理SaaS開発】（2017/07～2019/02）	【主軸】	【規模】

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
社会福祉協議会向けの相談・自立支援業務管理SaaSの開発に参画。相談記録、承認フロー、通知、権限管理、帳票出力をLaravelで実装し、Vue.js管理画面と連携することで、紙・Excel中心だった相談・支援業務のWeb化を支援。 【業務内容】 ・ PHP / Laravel による相談記録、承認、差戻し、通知処理の実装 ・ Vue.js を用いた管理画面、入力フォーム、検索・絞り込み機能の開発 ・ MySQL テーブル設計、相談履歴・権限管理データの設計補助 ・ 部署・役職ごとの権限差分、代理承認、差戻し時の通知条件整理 ・ Excel / PDF帳票出力、テンプレート調整、印刷レイアウト確認 ・ 単体テスト、結合テスト、顧客受入テスト時の不具合修正 ・ 利用部門からの問い合わせをもとにした入力補助、エラー文言の改善 【課題・工夫】 相談・支援業務の承認フローは部署・役職・支援種別によって分岐が多く、仕様変更の影響範囲が広い機能でした。条件分岐をコードだけで追うと保守が難しくなるため、承認ルート、通知タイミング、差戻し時の状態を表形式で整理し、テストケースへ反映しました。帳票出力では、画面上のデータと印刷物の見え方が一致するよう、テンプレート差分を細かく確認しました。 【工程別の担当】 要件整理：紙・Excelで運用されていた相談記録、承認、差戻し、帳票出力の流れを確認。 設計：承認ルート、権限、通知、履歴、帳票テンプレートのデータ構造を整理。 実装：Laravelでの承認処理、Vue.js入力フォーム、検索画面、通知処理、帳票出力を担当。 テスト：部署・役職・支援種別ごとの承認ルート、差戻し、代理承認、帳票レイアウトを確認。 受入対応：利用部門からの指摘をもとに入力補助、エラー文言、帳票フォーマットを調整。 【実績】 相談履歴と通知状態を画面上で確認できるようにし、運用担当者の問い合わせ対応を支援。紙・Excel運用で発生していた確認漏れを減らすため、承認状態、差戻し理由、通知履歴を一元化し、月次の確認作業を約10時間削減。帳票出力のフォーマット差分も吸収し、既存業務からの移行後の差戻し問い合わせを約30%削減した。	PHP, Laravel, Vue.js, MySQL	6名 【役割】 メンバー / 機能担当 【従事割合】 約60%
【医療機関向け施設基準管理システム改修】（2016/02～2017/06） 医療機関向け施設基準管理システムの改修PJに参画。施設基準一覧・詳細、従事者管理、申請書類出力、適時調査対応機能を対象に、フロントエンド改修とLaravel API開発を担当し、既存システムからの段階的な移行を支援。先輩エンジニアの指導のもと、実装補助・テスト・不具合修正を中心に担当。 【業務内容】 ・ Reactを用いた施設基準一覧、検索、詳細画面のフロントエンド改修補助 ・ Laravelによる従事者情報API、申請書類出力APIのテストおよび軽微な改修 ・ MySQLクエリ最適化補助、施設基準検索・従事者確認処理のパフォーマンス改善 ・ 既存画面との仕様差分確認、単体テスト・結合テスト項目の作成補助 ・ 申請期限切れ、従事者不足、重複登録、差戻し時の例外処理実装補助 ・ 一覧のページング、帳票プレビュー、キャッシュ制御の改善補助 ・ デザイナー、バックエンド担当との画面仕様・API仕様調整補助 ・ リリース前のシナリオテスト、申請導線確認、不具合修正 【課題・工夫】 施設基準管理では、検索、従事者確認、申請書出力、提出完了までの導線に少しでも分かりにくい箇所があると業務負担につながるため、正常系だけでなく、期限切れ、従事者不足、セッション切れ、二重送信などの例外系を重点的に確認しました。 【工程別の担当】 仕様確認：先輩エンジニアが整理した仕様をもとに、画面挙動を確認。 実装補助：Reactコンポーネント修正、Laravel APIの一部改修、バリデーション追加、エラー文言調整を担当。 テスト：申請期限切れ、従事者不足、重複登録、セッション切れ、二重送信、差戻しなどのテストを実施。 リリース後対応：既存システムから段階的に切り替えた後、表示速度と申請導線の不具合修正を継続。 【実績】 施設基準一覧周辺の表示速度改善、既存不具合の修正に貢献（当時のチーム計測によると表示時間は約2.4秒から1.6秒前後に改善）。申請・従事者管理まわりの例外処理整理により業務担当者からのエラー問い合わせを約15%削減。レビューを通じて、利用者導線、例外処理、テスト観点を意識した実装を習得。	【主軸】 PHP, Laravel, React, MySQL	【規模】 7名 【役割】 メンバー（実装・テスト補助） 【従事割合】 約70%
【医療・福祉業務システム開発補助】（2015/10～2016/01） 医療・福祉向け業務管理システムの開発補助・テスト補助を担当。先輩エンジニアの指導のもと、相談記録帳票出力の軽微な修正、入力チェック修正、不具合調査、テストデータ作成を通じて、実務開発の基礎を習得。 【業務内容】 ・ 相談記録・申請書類の帳票出力機能の軽微な修正、Excel / PDFフォーマット調整補助 ・ 入力チェック、エラーメッセージ、相談記録一覧検索条件の小規模改修 ・ Python / JUnitによる単体テスト補助スクリプト作成 ・ テストデータ作成、障害票調査、修正内容の確認 ・ 画面仕様書、テスト手順書、修正履歴のドキュメント更新 【課題・工夫】 初期の開発経験であったため、要件整理や設計判断は担当せず、まずは報告・相談、レビュー指摘の反	【補助】 Java, Spring, MySQL（インターン期）	【規模】 4名 【役割】 インターン / 開発補助 【従事割合】 約40%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
映、再現手順の記録を正確に行うことを重視しました。 【工程別の担当】 実装補助：帳票、入力チェック、一覧検索など小規模改修を先輩の指示に基づいて担当。 テスト補助：テストデータ作成、単体テスト、障害票の再現確認を実施。 ドキュメント：修正履歴、テスト手順、確認結果を更新し、レビュー時に説明できるようにした。 【実績】 テスト補助と手順書更新により、帳票まわりの回帰確認項目を約25件整理し、繰り返し確認作業の一部効率化に貢献。インターン期間を通じて基本的な開発プロセス、レビュー対応、報告・相談の進め方を習得。		

【退職理由】 医療・福祉向けソフトウェア開発で身につけた実装力を、自社サービスの継続改善に活かしたいと考え退職しました。

■テクニカルスキル

約10年の開発経験に基づくT字型スキルセットです。Ruby on Rails、MySQL、RSpec、AWS、GitHub Actionsを主軸とし、TypeScript / React / Vue.js、OpenAPI、Elasticsearch / Algolia、生成AI連携、BigQuery、Pythonを補助領域として位置づけています。

【主軸スキル】

種類	スキル	使用期間	レベル
言語	Ruby	約3年	Railsを用いたAPI、バッチ、RSpecテストの設計・実装・レビュー
FW	Ruby on Rails	約3年	REST API、認証、通知、バックグラウンドジョブ、DBマイグレーション
DB	MySQL	約9年	テーブル設計、SQL改善、一覧・検索の性能チューニング
DB	Aurora MySQL	約2年	AWS RDS上での運用、複合インデックス・クエリ改善
テスト	RSpec	約3年	モデル・APIの単体・結合テスト、CIへの組み込み
クラウド	AWS	約4年	ECS, RDS, S3, CloudFront 等を用いたWebサービス構築・運用
CI/CD	GitHub Actions	約6年	Lint、テスト、デプロイ自動化（CircleCI相当の運用経験あり）

【補助スキル】

種類	スキル	使用期間	レベル
言語	TypeScript / JavaScript	約8年	React / Next.js / Vue.js連携、管理画面・APIクライアント実装
FW	Laravel / Node.js	約6～7年	Railsと並行して利用したAPI・バッチ開発
フロントエンド	React / Vue.js	約3～4年	管理画面、既存機能改修、Rails API連携
検索	Elasticsearch / Algolia	約2年	一覧・検索APIの性能改善、導入検討・索引設計の補助
API	OpenAPI	約2年	仕様先行定義、フロント・モバイルとの開発並行
AI	OpenAI API（生成AI）	約2年	FAQ検索、回答候補、根拠リンク付き機能のプロダクト組み込み
分析	BigQuery	約1年	アクセス分析連携、集計要件の仕様整理・クエリ確認
言語	Python	約2年	データ整形スクリプト、検証用バッチ、Flask連携の補助
モバイル連携	REST API / BFF	約2年	モバイルアプリ向けAPI設計、Flutter / React Native連携
開発支援	GitHub Copilot	約2年	実装補助、PR説明、テスト観点作成（人がレビュー）
IaC / 環境	Terraform / Docker	約1～7年	AWS構成改修、ローカル・CI環境整備
DB	Redis	約2年	キャッシュ・セッション、API負荷軽減
コラボレーション	Slack / Notion / Figma	約4年	要件整理、画面確認、スプリント内の進捗共有

※上記は自己評価に基づくものです

■資格

取得年月	資格名	区分
2018年10月	基本情報技術者試験	IT基礎
2021年11月	AWS Certified Solutions Architect – Associate	クラウド
2023年07月	AWS Certified Developer – Associate	クラウド

取得年月	資格名	区分
2025年04月	HashiCorp Certified: Terraform Associate	laC

■得意領域・補助領域

主軸（T字の深い領域）：Ruby on Rails によるAPI・Webアプリケーション開発、MySQL の設計・チューニング、RSpec を用いたテスト、AWS + GitHub Actions によるリリース運用。2週間スプリントでの機能開発、コードレビュー、性能改善提案が中心です。

補助（T字の広い領域）：React / Vue.js、OpenAPI、Elasticsearch / Algolia、生成AI（OpenAI API）、BigQuery、モバイル向けAPI、Python、Slack / Notion / Figma。いずれもRailsバックエンドと組み合わせて利用してきました。

■技術経験補足（簡略）

【主軸】

RailsでのAPI設計・実装、MySQL性能改善、RSpec整備、AWS運用をキャリアの中心に据えています。物件・会員・受注データのように一覧・検索負荷が高いドメインでの改善経験があります。

【補助】

フロントエンド、Elasticsearch / Algolia、生成AI、BigQueryは主軸スタックを補完する形で約1～2年利用。SQLチューニングと検索基盤の導入検討・索引設計の補助経験があります。

■自己PR（失敗と学び）

【検索性能とユーザー体験】

分析SaaSで一覧・検索APIの応答が遅く、保存リストやCSV出力の操作が重いという問い合わせが増えました。SQL改善とRedisキャッシュ、検索基盤の導入検討を並行して進め、主要API応答時間を平均1.8秒前後から1.1秒前後へ改善。以降は「機能が動くこと」と「日常業務でストレスなく使えること」をセットで設計するよう意識しています。

【生成AI機能のプロダクト組み込み】

サポート向けAI検索では、回答文だけでなく参照元FAQと確認観点を併記する方針にしました。誤回答リスクを抑えつつ初動調査時間を短縮し、運用担当者のフィードバックを次の改善に反映するサイクルを回しました。空間データやバーチャル演出のような生成AI領域にも応用したいと考えています。

【今後取り組みたいこと】

不動産・住宅向けSaaSのように、360度コンテンツと物件データを安全に扱うバックエンド基盤の強化に携わりたいです。Rails 7系のアップグレードやSearchkick / Elasticsearch活用にも関心があり、スプリント開発の中でアーキテクチャと性能を継続改善したいと考えています。

以上